Experiment 6: Implementing Hebb's Rule

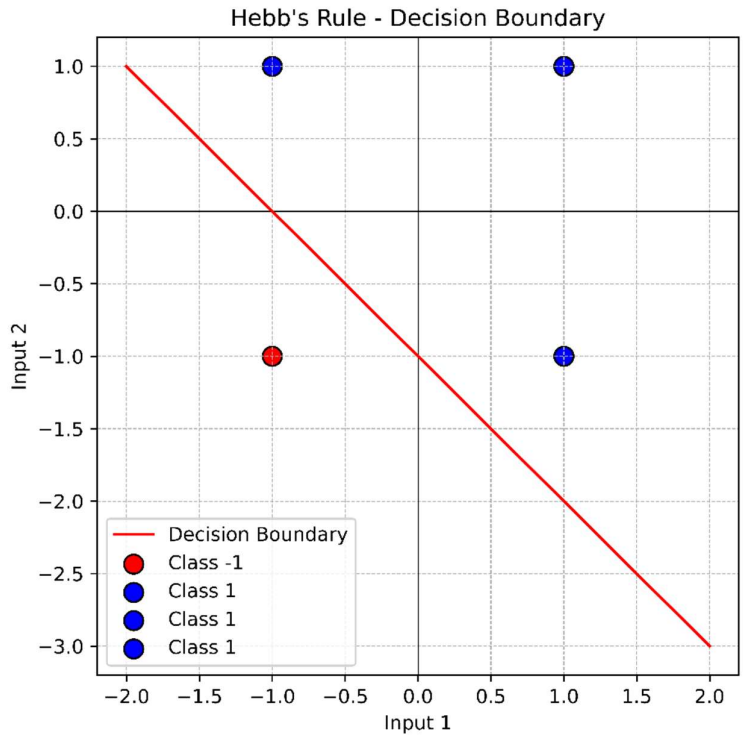
Question

To implement Hebb's learning rule and analyze weight adjustments based on inputs.

- **Final Weights and Bias**: The model learns weights based on Hebb's rule.
- **Decision Boundary**: The plotted line shows how the model classifies input patterns.

Implementation:

```
import numpy as np
import matplotlib.pyplot as plt

# Define input patterns and expected outputs
X = np.array([[-1, -1], [1, -1], [-1, 1], [1, 1]])
y = np.array([-1, 1, 1, 1]) # Target outputs

# Initialize weights and bias
weights = np.zeros(2) # Small random values to ensure updates
bias = np.zeros(1)
learning_rate = 1

# Hebbian Learning Rule
for i in range(len(X)):
    weights += learning_rate * X[i] * y[i]
    bias += learning_rate * y[i]

print('Final Weights:', weights)
print('Final Bias:', bias)

# Visualization of decision boundary only if weights are nonzero
if weights[1] != 0:
    x_vals = np.linspace(-2, 2, 100)
    y_vals = -(weights[0] * x_vals + bias) / weights[1]
    plt.figure(figsize=(6, 6))
    plt.axhline(0, color='black', linewidth=0.5)
    plt.axvline(0, color='black', linewidth=0.5)
    plt.grid(True, linestyle='--', linewidth=0.5)
    plt.plot(x_vals, y_vals, 'r-', label='Decision Boundary')

# Plot input points
for i in range(len(X)):
    plt.scatter(X[i][0], X[i][1], c='blue' if y[i] == 1 else 'red', marker='o', edgecolors='black', s=100, label=f'Class
{y[i]}')
plt.xlabel('Input 1')
plt.ylabel('Input 2')
plt.title("Hebb's Rule - Decision Boundary")
plt.legend()
plt.show()
```

Output

Final Weights: [2. 2.]
Final Bias: [2.]